Learned Dispatch for the Meridian eVTOL Network

Simulator, Independent Checker and Comparison with Greedy-JIT

Pankaj Kumar Barman

27 October 2026

1 Summary

In this assignment I built a simulator for the Meridian network, an independent checker for finished episodes, and a learned dispatch policy. I compared the policy with the given Greedy-JIT baseline. The main results are:

- **Simulator.** The simulator is event-driven, with one decision per request. It enforces energy, the 20% reserve, the charging curve, pad capacity, time windows and the 30-minute lead time. In its JIT mode it reproduces `greedy_jit.py` exactly (same schedule, same KPIs).
- **Checker.** A separate checker rebuilds every day from the CSV logs only. It checked 18,760 evaluation episodes and found **0 rule violations and 0 service failures**.
- **Policy.** A linear scoring policy over fully checked candidate plans, with hard action masking. It is trained by behaviour cloning of JIT followed by evolution strategies (ES) on the daily profit. Training took **2.2 minutes** on my laptop CPU.
- **Result.** On 200 held-out test days the policy earns **$4,499.7 per day against $4,056.3 for JIT (+10.9%)**. It is better on 93% of the days ($p = 1.2 \times 10^{-51}$, paired t-test). It stays better than JIT in all 25 stress settings I tested.
- **Main insight.** JIT loses most requests because of *battery charge*, not pads. Most of the gain comes from charging while pads are idle, plus declining a few low-margin requests.
- **Negative results.** On the single fixed 70-request instance the policy earns 2.6% *less* than JIT. A Monte-Carlo rollout teacher was clearly worse than JIT, and an acceptance-bonus reward gave no gain.

2 Problem A: Simulation environment

2.1 Modelling choice

The operator must accept or reject a request at the moment it becomes known, and an accepted plan cannot be changed later. So I used the reveal time of each request, $\max(0,\ \text{earliest_dep} - 30)$, as the decision epoch. Requests are handled in the order $(\text{earliest_dep}, \text{id})$, as in the baseline. Between

two decisions nothing has to be decided, because all aircraft movements are already fixed by earlier commitments. The time between decisions varies, so the problem is a semi-Markov decision process (SMDP).

- **State:** the new request; for each aircraft its location, the time it becomes free and its state of charge (SoC); the pad occupancy of each vertiport for every minute of the day.
- **Action:** `REJECT`, or "serve with aircraft k using charging mode m".
- **Transition:** the chosen plan is committed (its stints are booked on the pads) and the next request is revealed.
- **Reward:** the profit of the committed plan (Section 3.3).

2.2 Candidate plans and charging modes

For each aircraft and each mode, the planner builds one complete plan: pre-departure stint, optional deadhead, revenue flight and post-landing stint. Every plan is checked against all rules before it is offered. I used four modes:

Mode	Difference from JIT
JIT	None. An exact copy of `plan_for_aircraft` in `greedy_jit.py`.
PRECHARGE	The stint before departure (or before the deadhead) starts earlier, minute by minute, while a pad and the aircraft are free. It charges up to max(required SoC, 80%). The departure time is unchanged.
TOPUP	The stint after landing continues while a pad is free, up to 80% SoC.
LATE	The latest feasible departure in the window, instead of the earliest.

Why these modes. Before designing the actions, I checked why JIT rejects requests. For each forced JIT rejection on the 200 test days, I relaxed one resource at a time and tested whether any aircraft could then serve it:

Relaxed resource (6,129 JIT rejections, 200 test days)	Rejections that become servable
Battery: all aircraft at 100% SoC	4,830 (79%)
Aircraft time: no earlier commitment	4,279 (70%)
Pads: unlimited	3,211 (52%)

Battery charge is the most common bottleneck. Only 268 rejections can be fixed by more pads and by nothing else. JIT charges only the minimum and then parks the aircraft, where it cannot charge, so an aircraft is often close to the reserve when the next request comes. This is why I added modes that charge while a pad is idle.

2.3 How each requirement is met

Requirement	Implementation
Energy and reserve	SoC is a float and is never rounded. A plan is built only if the SoC before each flight is at least $20 + 12 + d$.
Charging curve	2%/min below 80%, 2/3%/min from 80% to 100%, capped at 100%, scaled by `charge_mult`.
Pad capacity	A minute-by-minute pad book. Every stint of a plan must fit.
Time windows	Departure inside the window. The post-landing stint must end by $t = 360$.
Lead time	No activity of a new plan starts before the reveal time.
Fixed instance and generator	The environment can replay `requests.csv` or sample a day by seed. The generator uses the same random stream as the baseline (same seed gives identical requests).
Determinism	Same seed and same actions give the same schedule (tested). Training with the same seed reproduced the same learning curve on the same machine.
Fidelity	JIT mode gives exactly the baseline schedule and KPIs on the fixed instance and on 15 seeds for each of 5 configurations (tested).

2.4 Independent constraint checker

The checker (`meridian/auditor.py`) reads only `requests.csv`, `schedule.csv`, `decisions.csv` and a small configuration file. I typed the vertiport coordinates in again from the scenario, and the checker recomputes distances, block times, energy and charging with its own code. It replays each aircraft from its start position at 90% SoC. It imports only the Python standard library (a test enforces this). It checks 14 rules:

- one decision per request with the correct reveal time;
- every accepted request has exactly one revenue flight (else it is a service failure); no rejected request is flown;
- the revenue flight has the right route and departs in the window;
- flight time $= 4 + \lceil d/3 \rceil$; all activities in $[0, 360]$; stints at least 5 minutes;
- correct start position, no overlaps, no movement while parked, every departure ends a stint and every landing starts one;
- SoC $\geq 20\%$ at every landing and never above 100%;
- pad occupancy $\leq$ pad count at every minute;
- no activity prepares a request before its reveal time;
- KPIs recomputed from the schedule match the reported KPIs.

To test the checker itself, I injected faults into a correct JIT log: a removed pad, a second aircraft on a single pad, a lower initial SoC, slower chargers, a deleted revenue flight, a rejected request that is flown, a shifted window, a shortened flight, a 3-minute stint, an aircraft that jumps between vertiports, and an activity before the reveal time. The checker caught every one. All 33 unit tests pass.

3 Problem B: Policy

3.1 Policy structure

Each available action gets a feature vector $\phi(s, c)$ with 111 entries:

- **plan features:** deadhead km, plan profit, departure delay, SoC at departure and at the end, minimum landing SoC, how long the aircraft is busy, stint lengths, whether the aircraft leaves its vertiport empty, pad use at the origin and at the destination after landing;
- **mode features:** the mode code and the product of each plan feature with the mode, so one feature can have a different effect in different modes;
- **context features:** time of day, expected demand, request margin, number of feasible aircraft, idle share, fleet SoC, pad scarcity and route. These are placed only on the `REJECT` row, so the reject threshold depends on the context.

The score is linear, $w^\top \phi(s, c)$, and the policy takes the arg-max over the *unmasked* actions only. The same weights are used for every candidate, so the policy does not depend on the fleet size. I could therefore evaluate it with 6 and 10 aircraft without retraining.

I did not start with this design. My first policy was a two-layer MLP. After behaviour cloning its hidden units were saturated: small ES noise changed no decision, and large noise changed almost all of them. ES had no useful signal. The linear model with mode interactions gives graded changes, and ES then improved it steadily. I also removed an "is this the JIT choice" feature, which made the score gaps too large for the same reason. Finally, in a first version the context features were on every row, so they cancelled out; moving them to the `REJECT` row fixed this.

3.2 Safety: hard action masking

I enforce safety **by construction**. The environment offers only plans that passed every check, all other actions are masked, and the environment raises an error if a masked action is passed. `REJECT` is always available. A plan is committed in full when it is accepted and is never changed, so an accepted request cannot fail later. A unit test runs a *random* policy over all unmasked actions and the checker finds no violation. In the full evaluation the checker found none in 18,760 episodes, including twice the demand, a closed vertiport and slow chargers.

I did not use the two alternatives:

- **Penalty in the reward.** A penalty makes violations rare, but does not rule them out. Under distribution shift the policy meets new states, and a learned trade-off between profit and penalty can still pick a violation. The size of the penalty is also hard to choose: too small and violations pay, too large and the policy rejects too much.
- **Shield that corrects actions afterwards.** The policy is then trained on actions that were not executed, which biases learning, and the correction rule is a hidden second policy that may be poor (for example, rejecting many good requests). Masking gives the same guarantee, and the policy learns the true effect of the safe options.

The limit of masking is that it is only as correct as the feasibility check. This is the reason for the separate checker.

3.3 Reward design

The main reward is the exact profit of the action:

$$r = \begin{cases} \text{rev} - 1.5\,(d_{\text{rev}} + d_{\text{dh}}) - 0.3\,(E_{\text{rev}} + E_{\text{dh}}) & \text{if accepted,} \\ 0 & \text{if rejected.} \end{cases}$$

The rewards of one day add up exactly to the day's profit, so the training objective is the business objective. I did not use a discount, because the day is short and finite. ES optimises the total return of a day, so a dense and a terminal-only reward give the same result here.

Rejected variant 1: acceptance bonus (+$50 per accepted request). This rewards the number of requests served, not their profit. I trained it with the same settings, and selected its model by its own reward to keep the comparison fair. On the test days it was close to the main policy (+$427.5 vs +$443.3 over JIT; the confidence intervals overlap). The reason is that every request has a positive direct margin ($36.4+2.2d > 0$), so the bonus mostly shifts the reject threshold. However, it was worse with the V1 pad outage (+$326 vs +$405), with fast chargers (+$132 vs +$160) and on the fixed instance ($3,857.88, the lowest of all policies). It also flew more deadhead (204.0 vs 194.4 km/day). It gives no gain, and it optimises a quantity that is not the objective, so I rejected it.

Rejected variant 2: shaping terms. I considered a bonus for SoC gained in the fast charging region and a penalty for pad congestion. Neither is potential-based, so both change the optimal policy. The SoC bonus pays for charge that may never be used. The policy already sees SoC and pad use as features, so I kept the true profit.

Rejected teacher: Monte-Carlo rollouts. I tried a teacher that scores each action by its immediate profit plus the profit of a base policy on 4 sampled future days. On 10 test days it earned $3,707.1 per day, against $4,029.2 for JIT and $4,305.3 for the precharge heuristic, and it took about 11 s per day. It declined 6–12 feasible requests per day. With so few samples, the best of about 9 noisy estimates is biased upwards (the optimiser's curse), so `REJECT` often wins by noise. More samples would help, but the cost grows linearly, so I did not use it.

3.4 Training

Stage	Method	Data
1. Behaviour cloning	Softmax cross-entropy to the JIT choice, Adam, L2 10^{-4}, 25 epochs	150 JIT days, 8,446 decisions
Calibration	Rescale the weights so the median score gap is 1	–
2. Evolution strategies	200 iterations, 16 antithetic pairs, $\sigma = 0.3$, Adam (lr 0.05), centred ranks	8 days per iteration, demand $\times U[0.8, 1.3]$, seeds 0–999, same days for all members
Model selection	Every 5 iterations	Validation seeds 5000–5029

Behaviour cloning reached 86.8% accuracy. Most mismatches are ties between plans that differ only in the aircraft index. The cloned policy earned $4,170.6 on the validation days, against $4,165.0 for JIT, so ES started from the baseline.

I chose ES because each decision is an arg-max over a candidate set whose size changes, the reward is naturally a whole-day number, and one day takes only a few tens of milliseconds to simulate. ES needs no gradient through the arg-max, no value function and no credit assignment, and it runs days in parallel. Using the same days for all population members (common random numbers) removes most of the day-to-day noise.

Hardware and time. Intel Core i9-14900 (24 cores, 32 threads), 16 GB RAM, Windows 11, Python 3.11, numpy only. Main policy: **132 s** (behaviour cloning 8 s). Bonus variant: 150 s. As a check on a slow machine, the same training took 17.2 min on a 2-core Intel Xeon VM. Both are far below the 45-minute limit.

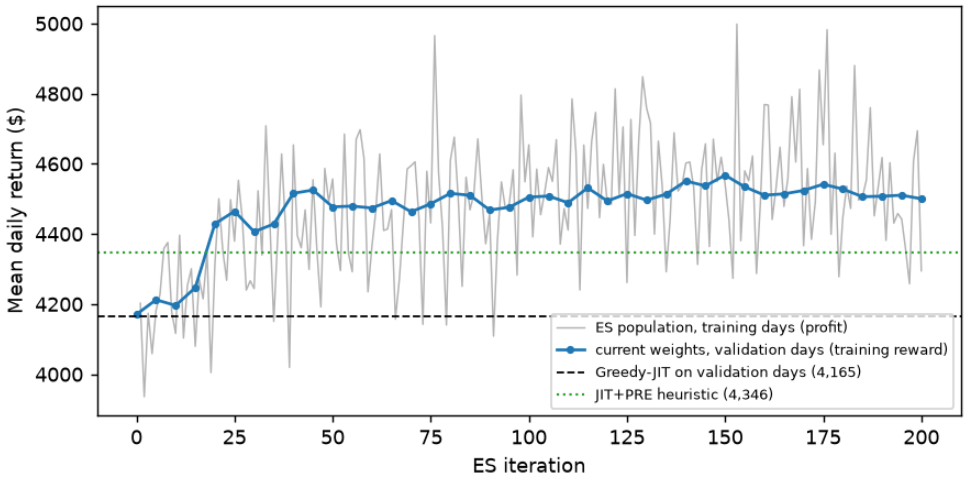


Figure 1: Learning curve. Validation profit rises quickly in the first 40 iterations and then slowly. The best checkpoint (iteration 150, $4,567.7 per day, +9.7% over JIT) is used.

4 Evaluation framework

- **Data split.** Training seeds 0–999, validation 5000–5029 (model selection only), test 2000–2199 (all reported results).
- **Paired design.** All policies run on the same 200 days. I report mean ± SD, median, the mean difference to JIT with a 95% confidence interval, the share of days better and worse, a paired t-test and a Wilcoxon signed-rank test.
- **Baselines.** Greedy-JIT, and a hand-written "JIT + precharge" rule (JIT+PRE). The second one shows how much a simple rule already gains.
- **Rejections.** Split into *forced* (no feasible plan) and *strategic* (feasible but declined).
- **Battery cycles.** Not defined in the scenario. I report the discharge equivalent full cycles (EFC = SoC used in flight/100, fleet per day), charge EFC, charge–discharge reversals, and landings below 30% SoC (deep discharge, the main cause of ageing).

- **Checker on everything.** Every episode of every policy and setting is re-checked.
- **Sensitivity, without retraining.** Demand × 0.7–2.0; demand peak shifted by −45 to +60 min; charging speed × 0.75–1.5; one pad removed at V0, V1 or V2, or V4 closed; 6 and 10 aircraft; and the fixed instance with all windows shifted by a random ±2, ±5 or ±10 minutes (30 variants each).

5 Results

5.1 Held-out test days (nominal scenario)

Metric	JIT	JIT+PRE	RL	RL-bonus
Profit, mean ± SD ($)	4,056.3 ± 395.6	4,315.1 ± 421.9	**4,499.7** ± 384.6	4,483.8 ± 376.0
Profit, median ($)	4,092.1	4,322.9	4,468.4	4,485.2
Difference to JIT [95% CI]	–	+258.8 [±34.5]	**+443.3** [±41.9]	+427.5 [±42.0]
Days better / worse than JIT	–	88% / 12%	93% / 7%	94% / 6%
Paired t-test p	–	1.4×10^{-33}	1.2×10^{-51}	2.7×10^{-49}
Requests per day	85.2	85.2	85.2	85.2
Served per day	54.56	57.83	58.98	58.83
Forced rejections per day	30.64	27.38	20.25	19.73
Strategic rejections per day	0.00	0.00	5.98	6.65
Service failures (total)	0	0	0	0
Deadhead km per day	218.5	232.1	194.4	204.0
Deadhead km per served request	4.04	4.05	3.32	3.50
Battery EFC, discharge (per day)	21.52	22.90	22.78	22.97
Battery EFC, charge (per day)	16.79	18.64	18.71	18.91
Charge–discharge reversals per day	66.5	70.5	70.1	70.5
Landings below 30% SoC per day	47.33	37.53	29.88	29.84
Min landing SoC, all days (%)	20.01	20.00	20.00	20.00
Pad use V0 / V1 / V2	.217/.315/.215	.239/.350/.245	.249/.365/.257	.252/.378/.264
Pad use V3 / V4	.431/.277	.458/.313	.403/.297	.382/.300
Inference time per day (ms)	15.3	55.9	64.8	64.0
Inference time per decision (ms)	0.18	0.66	0.76	0.75
Checker pass	200/200	200/200	200/200	200/200

Table 1: Comparison on 200 held-out test days (seeds 2000–2199).

My reading of these results:

- **Profit.** The learned policy earns $443 more per day than JIT (+10.9%) and $185 more than the simple precharge rule. So about 58% of the gain comes from charging earlier, which a hand-written rule can already do, and the rest is learned.
- **Requests served.** The policy serves 4.4 more requests per day even though it declines about 6 feasible ones. Its forced rejections drop from 30.6 to 20.3 per day: it keeps the fleet better charged and better placed.
- **Deadhead.** It flies 11% less deadhead per day, and 18% less per served request. The precharge rule alone does not reduce deadhead.

- **Battery.** The fleet flies more, so discharge EFC rises by 6%. But landings below 30% SoC fall by 37% (47.3 to 29.9 per day). The batteries work in a healthier range. The minimum landing SoC is still 20.00%, so some plans still use the full reserve margin.
- **Pads.** Pad use goes up at V0, V1 and V2 (more charging there), but down at V3 (0.431 to 0.403). The policy has learned to use the single pad at V3 less.
- **Inference time.** 0.76 ms per decision, against 0.18 ms for JIT, because the policy builds 4 modes $\times$ 8 aircraft plans per request. This is far below any real-time limit.

What the policy does. It uses PRECHARGE for 32–49% of accepted requests (most often for flights to V3, 49%), and it never uses TOPUP or LATE. Holding a pad after landing does not pay in this network; an "always TOPUP" rule earned about 17% less than JIT in my early tests. The 1,196 strategic rejections are cheap requests: the best plan declined had a median profit of \$45 (maximum \$89.7), and 61% of them were flights to V3.

5.2 Fixed 70-request instance

Policy	Profit (\$)	Served	Forced rej.	Strategic rej.	Deadhead km	EFC	Min SoC (%)
JIT	**4,011.64**	53	17	0	158.0	19.75	20.24
JIT+PRE	3,917.58	53	17	0	213.9	20.62	20.08
RL	3,907.98	52	14	4	167.6	19.64	20.08
RL-bonus	3,857.88	50	15	5	118.3	18.33	20.04

Table 2: Fixed instance. All four schedules pass the checker.

On this day JIT is best, and the learned policy loses 2.6%. It declined R14 (V1$\rightarrow$V3, 30 km deadhead, plan profit \$25.2), and two other requests that JIT served had become infeasible because of its earlier choices. The two extra requests it served earned only \$87.

I think this day is unusually good for JIT rather than typical. When I shift all windows of this instance by a few minutes (Table 3, jitter rows), JIT's mean profit drops to about \$3,715–3,770 and the learned policy is better by \$109–160 on average. The result on the exact fixed instance is therefore fragile, and a single day is not enough to judge a policy. I still report it, because it is the instance the assignment provides.

5.3 Sensitivity analysis

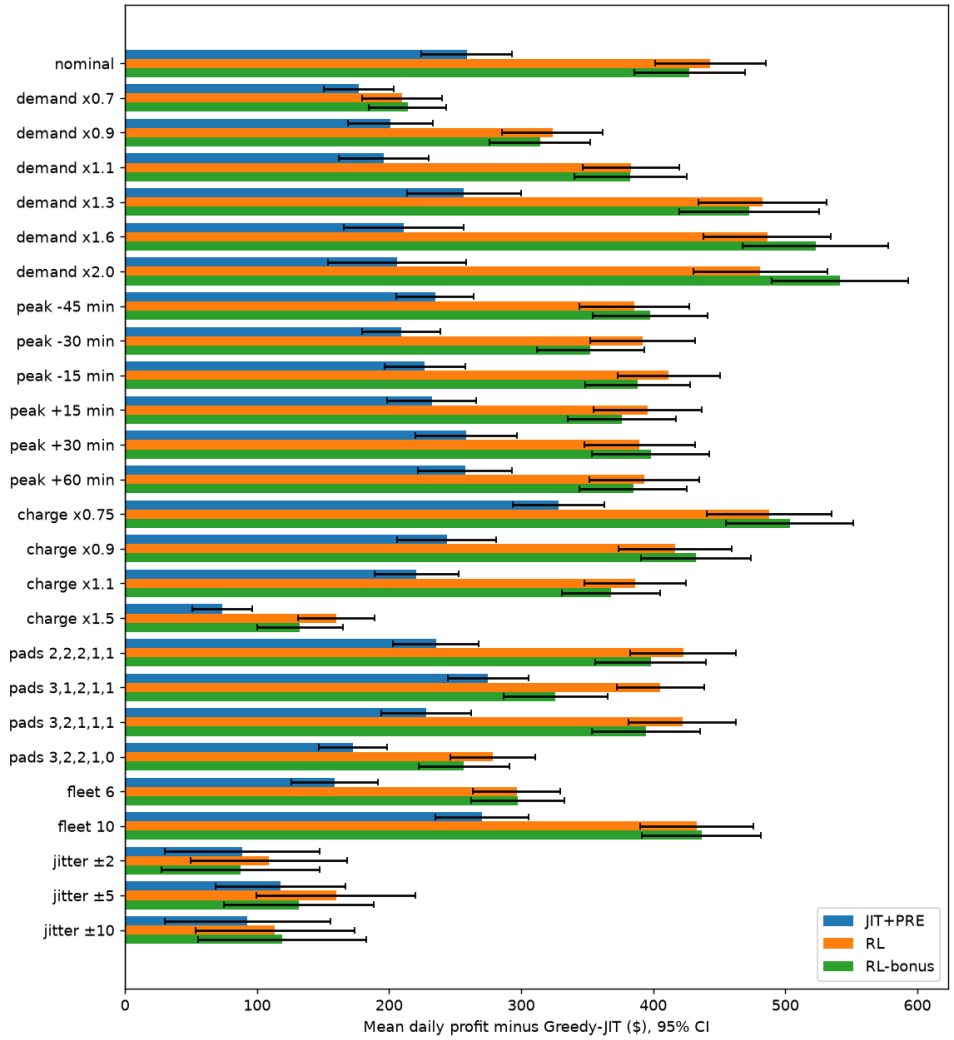


Figure 2: Mean daily profit minus JIT with 95% confidence intervals, for every stress setting (same trained weights, no retraining).

Setting	JIT profit	PRE − JIT	RL − JIT [95% CI]	RL gain	RL-bonus − JIT	Served JIT/RL
nominal	4,056.3	+258.8	+443.3 [±41.9]	10.9%	+427.5	54.6 / 59.0
demand ×0.7	3,396.6	+177.1	+209.9 [±30.1]	6.2%	+214.1	46.0 / 47.8
demand ×0.9	3,913.3	+201.1	+323.8 [±38.0]	8.3%	+314.2	52.6 / 55.6
demand ×1.1	4,322.4	+195.9	+383.4 [±36.6]	8.9%	+382.8	57.6 / 61.5
demand ×1.3	4,561.0	+256.6	+483.0 [±48.5]	10.6%	+472.6	60.7 / 65.6
demand ×1.6	4,948.8	+211.1	+486.3 [±48.2]	9.8%	+522.9	65.3 / 70.2
demand ×2.0	5,328.7	+206.0	+481.2 [±50.9]	9.0%	+541.7	69.5 / 74.6
peak −45 min	4,095.9	+234.8	+385.5 [±41.6]	9.4%	+397.6	55.1 / 58.8
peak −30 min	4,128.7	+209.1	+392.2 [±39.6]	9.5%	+352.5	55.5 / 59.1
peak −15 min	4,120.5	+226.9	+411.7 [±38.7]	10.0%	+388.1	55.3 / 59.3
peak +15 min	4,111.5	+232.4	+395.9 [±40.8]	9.6%	+376.3	55.2 / 59.0
peak +30 min	4,094.3	+258.3	+389.7 [±42.0]	9.5%	+398.1	55.1 / 58.9
peak +60 min	4,089.3	+257.5	+393.3 [±41.8]	9.6%	+384.8	54.9 / 58.7
charge ×0.75	3,488.4	+328.4	+487.9 [±47.4]	14.0%	+503.3	45.9 / 51.4
charge ×0.9	3,883.9	+243.6	+416.6 [±42.6]	10.7%	+432.4	51.8 / 56.1
charge ×1.1	4,247.7	+220.7	+386.2 [±38.6]	9.1%	+368.1	57.4 / 60.9
charge ×1.5	4,794.0	+73.7	+160.0 [±29.2]	3.3%	+132.4	65.0 / 65.6
pads 2,2,2,1,1	4,044.5	+235.6	+422.6 [±39.9]	10.4%	+398.0	54.3 / 58.3
pads 3,1,2,1,1	3,720.3	+275.0	+405.4 [±33.1]	10.9%	+326.1	50.1 / 54.0
pads 3,2,1,1,1	3,955.7	+228.2	+422.0 [±40.6]	10.7%	+394.4	53.1 / 56.9
pads 3,2,2,1,0	3,401.0	+172.8	+278.7 [±32.0]	8.2%	+256.8	46.9 / 50.0
fleet 6	3,357.4	+159.0	+296.6 [±33.0]	8.8%	+297.4	45.8 / 48.6
fleet 10	4,609.9	+270.3	+433.1 [±42.9]	9.4%	+436.5	61.2 / 65.4
fixed, jitter ±2	3,769.7	+88.8	+109.1 [±59.3]	2.9%	+87.7	50.2 / 50.9
fixed, jitter ±5	3,732.2	+117.5	+159.9 [±60.3]	4.3%	+131.5	49.9 / 51.3
fixed, jitter ±10	3,714.9	+92.8	+113.7 [±60.4]	3.1%	+119.0	49.6 / 50.1

Table 3: Sensitivity. 200 test days per setting (30 variants for the jitter rows). "RL gain" is RL − JIT as a share of JIT profit.

What I take from the sensitivity runs:

- **The gain holds in every setting.** The learned policy beats JIT in all 25 stress settings, and every confidence interval excludes zero. This includes settings far outside training: demand ×2.0 (training used ×0.8–1.3), fleets of 6 and 10 aircraft, and a closed vertiport.
- **The gain follows the battery bottleneck.** It is largest with slow chargers (×0.75: +14.0%) and smallest with fast chargers (×1.5: +3.3%). When charging is fast, JIT's just-in-time charging is almost good enough. This supports the diagnosis in Section 2.2.
- **Demand.** In absolute terms, the gain grows with demand up to ×1.3 and then stays flat at about $480–490 per day. At low demand (×0.7) the gain is only 6.2%, and the policy is only $33 better than the precharge rule. Here it serves fewer requests than the precharge rule (47.8 vs 48.3): with spare capacity, declining requests is not useful.
- **Peak shift.** The gain stays between $386 and $412 for all shifts. The policy uses the *nominal* demand curve as a feature, but a shift of up to one hour does not hurt it much.
- **Pads and fleet.** Removing one pad at V0, V1 or V2 changes little. Closing V4 reduces the gain to 8.2%. The fleet results show that the shared scorer transfers to other fleet sizes.

- **Small input noise.** On the jittered fixed instance the gain is positive but smaller (2.9–4.3%) and less certain (wide intervals, only 30 variants).

6 Failure cases and deployment risk

6.1 Concrete failure cases

The policy earns less than JIT on 14 of the 200 test days. I looked at the three worst days in detail.

1. **Earlier choices block later requests (seed 2082, −$547).** The policy declined only 1 request, but 12 requests that JIT served had become infeasible for it, and it flew more deadhead (283.9 vs 235.7 km). Its earlier assignments left aircraft in the wrong places for the later demand. A policy that scores one decision at a time cannot see this chain of effects.
2. **Strategic rejections that do not pay (seed 2000, −$138.6; seed 2077, −$240.7).** On seed 2000 it declined 8 requests that JIT served, mostly to V3 and V4 with 19–24 km of deadhead. The aircraft it saved earned $686 on 9 other requests, which was not enough. On seed 2077 one rejection was sensible (37.2 km deadhead for $5 profit), but others were requests with $39–65 profit.
3. **Bias against V3.** 735 of the 1,196 strategic rejections (61%) are flights to V3, which has one pad. On average this is right, but it is wrong on days with good return demand from V3.
4. **Low demand.** At demand ×0.7, rejecting is rarely useful, and the policy is barely better than the precharge rule.
5. **The fixed instance** (Section 5.2).

6.2 Where it would be unsafe to deploy

No rule was broken in any simulated episode. Even so, I would not deploy this policy as it is, because the guarantee only holds inside the model.

- **Deterministic physics.** Flight time, energy use and charging are exact in the model. Real wind, payload, temperature and battery ageing change them. The minimum landing SoC over all test days was 20.00%: the policy uses the full reserve margin. A small extra energy use would put an aircraft below the reserve. A real system needs an extra margin and a check just before each departure.
- **No re-planning.** Plans are fixed when a request is accepted. One late landing can block a single pad (V3 or V4) and break every later plan that counted on it. A real system needs online re-planning and a fallback for committed plans that become infeasible.
- **Equipment faults.** Charger or pad faults during the day are not modelled. A fault after acceptance would cause service failures that the policy cannot avoid.
- **Demand model.** The policy was trained on one Poisson generator. Real demand (events, weather) can be very different, and then strategic rejections can lose money. It should be tested on real booking data and watched for drift.

7 Conclusion

A learned policy can improve on hand-written JIT dispatch in this network, by about 11% on average, and the improvement holds under all the stress settings I tried. Most of the gain is simple to explain: the battery is the main bottleneck, so charging while pads are idle helps. The learned part adds selective rejection of low-margin requests and fewer deadhead flights. The same selective rejection also causes the largest losses on the days when it is wrong, including the fixed instance. Safety does not depend on learning: it comes from masking over fully checked plans, and an independent checker confirms it.

AI-tool disclosure

I first used Gimini Argon to generate state-of-the-art multi-agentic prompts to write a strategy to solve this problem modified as needed. Then I used Claude (Anthropic Opus 5.5 Medium) as a coding assistant during this assignment. It wrote the first version of most of the code (simulator, checker, policy, training and evaluation scripts, and unit tests) from the assignment files and my instructions, and it helped draft this report. I set up the environment and ran all training and evaluation on my own machine, dealt with a Windows file-encoding issue in the output scripts, and went through the result files to write the interpretation. I have checked the design decisions and can explain or change any part of the submission.

System used to run code:
Device name DESKTOP-0I2EQG2
Processor Intel(R) Core(TM) i9-14900 (2.00 GHz)
Installed RAM 16.0 GB (15.7 GB usable)
System type 64-bit operating system, x64-based processor